Assignment 18.4

Task 1 – Movie Database API Integration

Scenario

You are building a movie recommendation tool that fetches movie details from a public Movie Database API.

```python
#Generate a Python script using the requests library to fetch movie details from a Movie API (like OMDb) using a user-provided title.
# Display Title, Year, Rating, and Plot. Include error handling for invalid movie names, API key errors, network issues, and empty responses.
# Keep the code clean and readable.
import requests
def fetch_movie_details(title, api_key):
    url = f"http://www.omdbapi.com/?t={title}&apikey={api_key}"
    try:
        response = requests.get(url)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()

        if data['Response'] == 'False':
            print(f"Error: {data['Error']}")
            return

        print(f"Title: {data.get('Title', 'N/A')}")
        print(f"Year: {data.get('Year', 'N/A')}")
        print(f"Rating: {data.get('imdbRating', 'N/A')}")
        print(f"Plot: {data.get('Plot', 'N/A')}")

    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err}")
    except requests.exceptions.ConnectionError as conn_err:
        print(f"Connection error occurred: {conn_err}")
    except requests.exceptions.Timeout as timeout_err:
        print(f"Timeout error occurred: {timeout_err}")
    except requests.exceptions.RequestException as req_err:
        print(f"An error occurred: {req_err}")
if __name__ == "__main__":
    api_key = input("Enter your OMDb API key: ")
    movie_title = input("Enter the movie title: ")
    fetch_movie_details(movie_title, api_key)
```

```
Enter your OMDb API key: 35ef1330
Enter the movie title: Interstellar
Title: Interstellar
Year: 2014
Rating: 8.7
Plot: When Earth becomes uninhabitable in the future, a farmer and ex-NASA pilot, Joseph Cooper, is tasked to pilot a spacecraft, along with a team of researchers, to find a new planet for humans.
```

Task 2 – Public Transport Schedule API

Scenario

A city transport app needs to fetch bus/train arrival timings from a transport API.

```
#Generate a Python script using requests to fetch next 3 transport arrival times using a stop ID.
#Include input validation, retry logic, and error handling.
```

```python
import requests
import time
BASE_URL = "https://api-v3.mbta.com/predictions"
def fetch_transport_arrival_times(stop_id):
    retries = 2
    for attempt in range(retries + 1):
        try:
            params = {
                "filter[stop]": stop_id,
                "sort": "arrival_time"
            }
            response = requests.get(BASE_URL, params=params, timeout=5)
            response.raise_for_status()
            data = response.json()
            # Check if data is empty
            if not data.get("data"):
                print("❌ Invalid stop ID or no arrivals found.")
                return []
            arrivals = []
            for item in data["data"]:
                arrival_time = item["attributes"]["arrival_time"]
                if arrival_time:
                    arrivals.append(arrival_time)
                    if len(arrivals) == 3:
                        break
            return arrivals
        except requests.exceptions.Timeout:
            print(f"⏱ Attempt {attempt+1}: Timeout error")
        except requests.exceptions.ConnectionError:
            print(f"⚠ Attempt {attempt+1}: Server unavailable")
        except requests.exceptions.RequestException as e:
            print(f"❌ Error: {e}")
            return []
        time.sleep(1)
    print("❌ Failed after retries.")
    return []
def main():
    stop_id = input("Enter stop ID (e.g., place-sstat): ").strip()
    if not stop_id:
        print("❌ Stop ID cannot be empty.")
        return
    arrivals = fetch_transport_arrival_times(stop_id)
    if arrivals:
        print("\n🚌 Next 3 Arrivals")
        print("-" * 30)
        for t in arrivals:
            print(t)
        print("-" * 30)
    else:
        print("No arrival times available.")
if __name__ == "__main__":
    main()
```

```
Enter stop ID (e.g., place-sstat): North Station
❌ Invalid stop ID or no arrivals found.
No arrival times available.
```

Task 3 – Stock Market Data API

Scenario

You are developing a finance dashboard that retrieves live stock prices.

Task

Use AI to generate a Python script that fetches stock data from a financial API.

```
#Generate a Python script using the requests library to fetch stock market data from a financial API (like Alpha Vantage).
#Accept stock symbol as user input and display current price, day high, and day low.
#Include error handling for invalid symbols, API rate limits, network issues, and missing JSON fields.
#Keep the code clean and readable.
```

```python
import requests
def fetch_stock_data(symbol):
    api_key = "YOUR_ALPHA_VANTAGE_API_KEY"
    url = "https://www.alphavantage.co/query"
    try:
        params = {
            "function": "GLOBAL_QUOTE",
            "symbol": symbol,
            "apikey": api_key
        }
        response = requests.get(url, params=params, timeout=5)
        response.raise_for_status()
        data = response.json()
        # Handle API rate limit
        if "Note" in data:
            print("⚠ API rate limit reached. Try again later.")
            return
        # Validate response
        if "Global Quote" not in data or not data["Global Quote"]:
            print("❌ Invalid stock symbol or no data found.")
            return
        quote = data["Global Quote"]
        # Safely extract values
        current_price = quote.get("05. price", "N/A")
        day_high = quote.get("03. high", "N/A")
        day_low = quote.get("04. low", "N/A")
        print("\n📊 Stock Details")
        print("-" * 30)
        print(f"Symbol        : {symbol.upper()}")
        print(f"Current Price : {current_price}")
        print(f"Day High      : {day_high}")
        print(f"Day Low       : {day_low}")
        print("-" * 30)
    except requests.exceptions.HTTPError as http_err:
        print(f"❌ HTTP error: {http_err}")
    except requests.exceptions.ConnectionError:
        print("❌ Connection error. Check your internet.")
    except requests.exceptions.Timeout:
        print("⚠ Request timed out. Try again.")
    except requests.exceptions.RequestException as req_err:
        print(f"❌ Request error: {req_err}")
    except ValueError:
        print("❌ JSON decoding error.")
    except Exception as e:
        print(f"❌ Unexpected error: {e}")
# Main Program
if __name__ == "__main__":
    stock_symbol = input("Enter stock symbol (e.g., AAPL): ").strip()
    if not stock_symbol:
        print("❌ Symbol cannot be empty.")
    else:
        fetch_stock_data(stock_symbol)
```

```
Enter stock symbol (e.g., AAPL): AAPL
Enter stock symbol (e.g., AAPL): AAPL


📊 Stock Details
------------------------------
📊 Stock Details
------------------------------
Symbol        : AAPL
------------------------------
Symbol        : AAPL
Current Price : 252.6200
Symbol        : AAPL
Current Price : 252.6200
Day High      : 255.0000
Current Price : 252.6200
Day High      : 255.0000
Day Low       : 251.6000
Day High      : 255.0000
Day Low       : 251.6000
Day Low       : 251.6000
------------------------------
```

Task 4 – Geolocation API (IP Address

Lookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user

based on IP address.

```
#Write a Python script that takes an IP address, validates it, and fetches its country, city, and ISP using a Geolocation API.
#Include error handling for invalid IPs, API failures, and JSON errors.
```

```python
import requests
import re
def validate_ip(ip):
    # Regular expression for validating an IP address
    pattern = re.compile(r"^(?:[0-9]{1,3}\.){3}[0-9]{1,3}$")
    if pattern.match(ip):
        return all(0 <= int(num) <= 255 for num in ip.split('.'))
    return False
def fetch_geolocation(ip):
    url = f"http://ip-api.com/json/{ip}"
    try:
        response = requests.get(url)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()
        if data['status'] == 'success':
            return {
                'country': data.get('country', 'N/A'),
                'city': data.get('city', 'N/A'),
                'isp': data.get('isp', 'N/A')
            }
        else:
            raise ValueError("API returned an error: " + data.get('message', 'Unknown error'))
    except requests.exceptions.RequestException as e:
        print(f"API request failed: {e}")
    except ValueError as e:
        print(e)
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
    return None
if __name__ == "__main__":
    ip_address = input("Enter an IP address: ")
    if validate_ip(ip_address):
        geolocation = fetch_geolocation(ip_address)
        if geolocation:
            print(f"Country: {geolocation['country']}")
            print(f"City: {geolocation['city']}")
            print(f"ISP: {geolocation['isp']}")
    else:
        print("Invalid IP address format.")
```

```
Enter an IP address: 142.250.190.78
Country: United States
Country: United States
City: Chicago
ISP: Google LLC
```

Task 5 – Real-Time Application: Payment

Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a

sandbox payment API.

```
#Generate a Python script that simulates payment processing using a sandbox API. The script should:
#Accept amount, currency, and payment method as input.
#Validate inputs.
#Send a POST request to a test payment API endpoint.
#Handle invalid payment details, HTTP errors (400, 401, 500), and network errors.
#Display transaction status.
#Log failed transactions to a file.
```

```python
import requests
import logging
# Configure logging
logging.basicConfig(
    filename='payment_errors.log',
    level=logging.ERROR,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
# Input validation
def validate_inputs(amount, currency, payment_method):
    if amount <= 0:
        raise ValueError("Amount must be greater than zero.")
    if currency not in ['USD', 'EUR', 'GBP']:
        raise ValueError("Currency must be USD, EUR, or GBP.")
    if payment_method not in ['credit_card', 'paypal', 'bank_transfer']:
        raise ValueError("Payment method must be credit_card, paypal, or bank_transfer.")
# Simulate payment processing
def process_payment(amount, currency, payment_method):
    try:
        validate_inputs(amount, currency, payment_method)

        # Simulate POST request (using httpbin.org for testing)
        response = requests.post('https://httpbin.org/post', json={
            'amount': amount,
            'currency': currency,
            'payment_method': payment_method
        })
        response.raise_for_status()  # Raise error for bad responses
        print("Transaction successful!")
        print("Response:", response.json())  # Show sandbox response
    except ValueError as ve:
        logging.error(f"Validation error: {ve}")
        print(f"Validation error: {ve}")
    except requests.exceptions.HTTPError as he:
        logging.error(f"HTTP error: {he}")
        print(f"HTTP error: {he}")
    except requests.exceptions.RequestException as re:
        logging.error(f"Network error: {re}")
        print(f"Network error: {re}")
# Main program
if __name__ == "__main__":
    amount = float(input("Enter the amount: "))
    currency = input("Enter the currency (USD, EUR, GBP): ")
    payment_method = input("Enter the payment method (credit_card, paypal, bank_transfer): ")
    process_payment(amount, currency, payment_method)
```

```
Enter the amount: 100000
Enter the currency (USD, EUR, GBP): USD
Enter the payment method (credit_card, paypal, bank_transfer): credit_card
Transaction successful!
Response: {'args': {}, 'data': '{"amount": 100000.0, "currency": "USD", "payment_method": "credit_card"}', 'files': {}, 'form': {}, 'headers': {'Accept': '*/*', 'Accept-Encoding': 'gzip, deflate', 'Content-Length': '72', 'Content-Type': 'application/json', 'Host': 'httpbin.org
', 'User-Agent': 'python-requests/2.33.0', 'X-Amzn-Trace-Id': 'Root=1-69c4f0a5-17c0d68c0f23d0422ffe1f7b'}, 'json': {'amount': 100000.0, 'currency': 'USD', 'payment_method': 'credit_card'}, 'origin': '112.133.216.160', 'url': 'https://httpbin.org/post'}
```